

Experiment 8: Insertion Sort with Duplicate Elements

Aim: To implement Insertion Sort on an array containing duplicate elements and analyze its time complexity.

Algorithm

1. Read the array containing duplicate elements.
2. Consider the second element as the key.
3. Compare the key with elements before it.
4. Shift elements greater than the key one position to the right.
5. Insert the key into its correct position.
6. Repeat until all elements are sorted.
7. Display the sorted array.

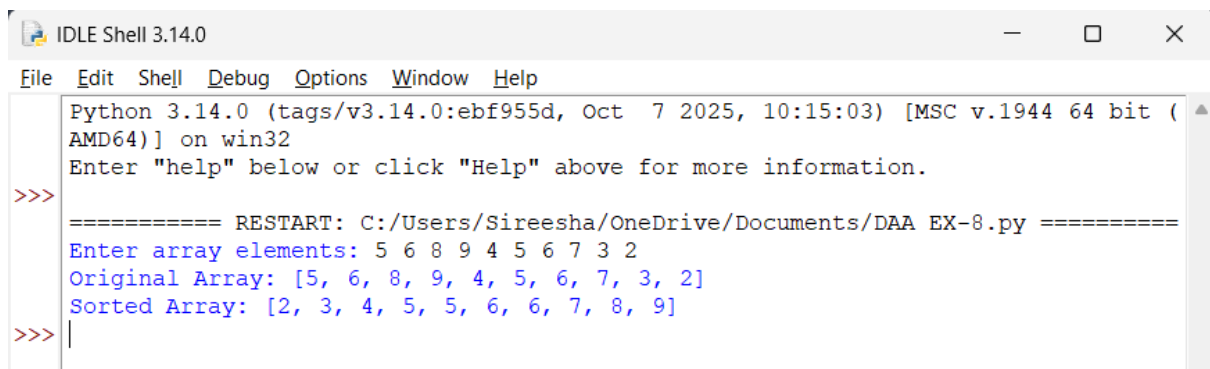
Python Program

```
def insertion_sort(arr):  
    n = len(arr)  
    for i in range(1, n):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j = j - 1  
        arr[j + 1] = key  
arr = list(map(int, input("Enter array elements: ").split()))  
print("Original Array:", arr)  
insertion_sort(arr)  
print("Sorted Array:", arr)
```

Input

Enter array elements: 5 6 8 9 4 5 6 7 3 2

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-8.py =====
Enter array elements: 5 6 8 9 4 5 6 7 3 2
Original Array: [5, 6, 8, 9, 4, 5, 6, 7, 3, 2]
Sorted Array: [2, 3, 4, 5, 5, 6, 6, 7, 8, 9]
>>>
```

Result

Thus, the Insertion Sort algorithm was successfully implemented for an array containing duplicate elements, and the array was sorted in ascending order.